

Figure 1 shows a sample OpenAPI workflow. Here some Go code is auto-generated from the API spec *api.yml*, which is combined with manually written Go code (kept in a separate file) to build the final server binary. The same YAML file can be fed into another tool to generate the HTML documentation.

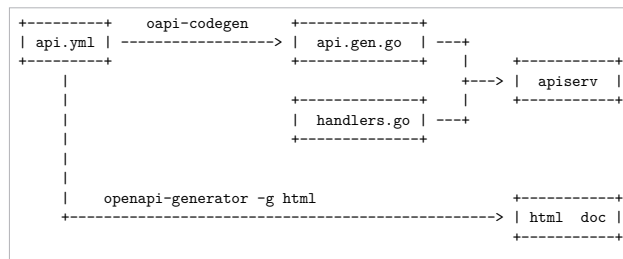


Figure 1: OpenAPI workflow

Here is a part from the YAML input, which is a spec for an API with two endpoints to find the sum and difference of two numbers:

```

/calc/sum:
  get:
    description: Returns the sum of the given numbers.
    parameters:
      - name: x
        in: query
        required: true
        schema:
          type: integer
      - name: "y"
        in: query
        required: true
        schema:
          type: integer
    # "y" needs quoting because y is boolean yes in YAML
    responses:
      '200':
        description: The sum.

```

Here are the handwritten handlers in Go, which perform the actual logic:

```

func (calc Calc) GetCalcSum(w http.ResponseWriter,
    r *http.Request,
    params GetCalcSumParams) {
    fmt.Fprintf(w, "%d\n", params.X + params.Y)
}
func (calc Calc) GetCalcDiff(w http.ResponseWriter,
    r *http.Request,
    params GetCalcDiffParams) {
    fmt.Fprintf(w, "%d\n", params.X - params.Y)
}

```

That's it. The YAML file has 43 lines, and handwritten Go code has less than sixty lines. *oapi-codegen* generated 351 lines that contain parameter extraction, validation, error messages, etc. Here are the auto-generated validation and error messages in action:

```

$ curl 'localhost:8080/calc/diff?y=12'
parameter "x" in query has an error: \
value is required but missing
$ curl 'localhost:8080/calc/diff?x=10&y=someString'
parameter "y" in query has an error: \
value someString: an invalid integer: invalid syntax

```

Example: enum stringification in C

Many times we need to print the name of an enum member in C, usually for logging or debugging purposes. But such information is not available to the program at runtime. So we end up writing an array that contains string constants representing the enum members, like this:

```

// XXX Manually keep in sync with the array opstr
typedef enum Operation {
    OP_SUM,
    OP_DIFF,
    OP_LARGEST,
    OP_SMALLEST,
} Operation;
// XXX Manually keep in sync with the enum
const char * opstr[] = {
    "OP_SUM",
    "OP_DIFF",
    "OP_LARGEST",
    "OP_SMALLEST" };

```

Now *opstr[OP_LARGEST]* would give us the string *"OP_LARGEST"*.

But see how we had to put a comment asking future maintainers to keep the blocks in sync? What if they (read: us) missed those comments? That would depend on the mistake. If changes were made in the beginning or middle for either structure, we'd get incorrect logs, without ever realising it. If the enum got a new value *OP_DIV* at the end and the array was never updated, our program would probably crash with a *SEGV* (because *opstr[OP_DIV]* points to invalid memory area now).

Now let's write a Makefile that invokes *grep* and *sed* to automate the generation of the array.

```

opstr.gen.h: main.c
    (echo '// Do not edit this file!' &&\
    echo 'const char * opstr[] = { ' &&\
    grep -E '^s*OP_[A-Z]+,$$' main.c)\

```